PUSL2020 - Software Development Tools and Practices

Project Proposal Compliance Service

Technical Report for Referral Coursework 25-26

Student: Navida Perera

Deliverable: ASP.NET Core Web API, premium web console, automated tests, and submission package

Technology	**ASP.NET Core 9 Web API**
Architecture	Controller, request model, compliance service, response contracts
Endpoint	POST /api/proposal/analyse
Quality Evidence	Build passed with 0 warnings/errors; 8 automated tests passed

Table of Contents

1. Introduction

The Project Proposal Compliance Service is a Web API that screens incoming project proposal submissions before they enter the Project Approval System blind-matching pipeline. It identifies basic data quality issues and early compliance concerns around budget, specialist resource allocation, and ethics region risk.

The solution includes a premium browser console for demonstration, but the core assessed service is the ASP.NET Core API. The API accepts structured proposal data, applies server-side validation, runs rule-based compliance analysis, and returns a stable JSON response for clients, testers, or future PAS workflow integration.

2. Development

2.1 System Overview

The system is intentionally simple and testable. The controller receives HTTP input, model validation protects the request contract, the compliance service owns rule analysis, and the response contract returns a consistent decision envelope.

Step	Component	Responsibility
1	Client / premium UI	Submits proposal JSON to the API endpoint.
2	ProposalController	Accepts the request body and returns the response envelope.
3	ProposalRequest model	Applies Data Annotation validation before compliance analysis.
4	ComplianceAnalysisService	Runs budget, resource, and ethics rules.
5	ProposalComplianceResponse	Returns status, validation errors, and compliance alerts.

2.2 Model and Validation

The incoming request is represented by ProposalRequest. Data Annotations are used so invalid submissions are rejected before compliance analysis. Two key examples are Range on Quantity and Range on EstimatedBudgetUSD, which ensure both values are greater than zero.

```
public sealed class ProposalRequest
{
    [Required(ErrorMessage = "Student name is required.")]
    [StringLength(120, MinimumLength = 2)]
    public string StudentName { get; init; } = string.Empty;

    [Range(1, 100, ErrorMessage = "Quantity must be greater than 0 and no more than 100.")]
    public int Quantity { get; init; }

    [Range(typeof(decimal), "0.01", "1 000 000", ErrorMessage = "Estimated budget must be greater than 0.")]
    public decimal EstimatedBudgetUSD { get; init; }
}
```

2.3 Compliance Logic

The compliance rules are implemented in ComplianceAnalysisService rather than inside the controller. This keeps the controller thin and makes the rules easy to test. The three rules are: high budget warning above $5,000, high resource allocation alert for Advanced GPU Cluster quantity greater than 3, and restricted ethics region alert for Bermuda.

```
if (proposal.EstimatedBudgetUSD > 5000m)
{
    alerts.Add(new ComplianceAlert(
```

```
        RuleCode: "BUDGET-001",
        Severity: ComplianceSeverity.Low,
        Message: "High funding limit detected. Requires manual academic coordinator review."));
}
```

2.4 API Response

Endpoint: POST /api/proposal/analyse. The response always includes complianceStatus, validationErrors, and complianceAlerts. This makes it predictable for both the premium UI and any automated client.

```
{
 "complianceStatus": "ReviewRequired",
 "validationErrors": [],
 "complianceAlerts": [
  {
   "ruleCode": "RESOURCE-001",
   "severity": "High",
   "message": "Potential resource monopolization alert for Advanced GPU Cluster."
  }
 ]
}
```

2.5 Setup Instructions

```
cd /path/to/submission
dotnet restore ProposalCompliance.sln
dotnet build ProposalCompliance.sln --no-restore
dotnet test ProposalCompliance.sln --no-build
dotnet run --project ProposalCompliance.Api
```

After running the API project, open the local URL shown in the terminal. The web console is served at the root path, and API requests are submitted to /api/proposal/analyse.

3. Testing and QA

3.1 Test Plan Summary

Testing focused on the service contract, model validation behavior, individual compliance rules, combined rule behavior, and API response shape. Automated tests use xUnit and Microsoft.AspNetCore.Mvc.Testing. Manual test design uses equivalence partitioning for valid/invalid ranges, boundary value analysis around the $5,000 threshold and quantity limit, and decision-table thinking for combined alerts.

3.2 Manual Test Results

ID	Scenario	Input Focus	Expected Result	Status
TC01	Low-risk valid proposal	Quantity 1, budget 1500, Sri Lanka	Approved; no alerts	Pass
TC02	Validation failure	Quantity 0 and budget 0	ValidationFailed; field errors	Pass
TC03	Budget rule	Budget 5000.01	BUDGET-001; Low	Pass
TC04	Resource rule	Advanced GPU Cluster, quantity 4	RESOURCE-001; High	Pass
TC05	Ethics rule	Bermuda	ETHICS-001; High	Pass
TC06	Combined risk	Budget 9000, GPU 4, Bermuda	Three alerts returned	Pass

3.3 Key Findings

All core coursework requirements are implemented. Build verification passed with zero warnings and zero errors. The automated suite passed 8 out of 8 tests. The validation response is structured, and all three compliance rules return explicit rule codes and severities. The premium UI is not required by the brief but improves demonstration quality and shows the JSON request contract clearly.

4. Conclusion

This project demonstrates a clean Web API implementation for initial proposal compliance screening. The design separates concerns between request validation, controller orchestration, compliance analysis, and response contracts. The solution is reliable for the defined coursework scope and is packaged with setup instructions, automated tests, manual test evidence, and a concise professional report.